IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 27 December 2024, accepted 5 February 2025, date of publication 11 February 2025, date of current version 18 February 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3540836

RESEARCH ARTICLE

Design and Implementation of High-Speed Carry Look-Ahead Decimal Adder (CLDA) Using CMOS Technology

ABDELSALAM AL SHARE[1], OSAMA AL-KHALEEL[2], FADI NESSIR ZGHOUL[1], MOHAMMAD AL-KHALEEL[3,4], AND CHRIS PAPACHRISTOU[5], (Life Fellow, IEEE)

[1]Electrical Engineering Department, Jordan University of Science and Technology, Irbid 22110, Jordan
[2]Computer Engineering Department, Jordan University of Science and Technology, Irbid 22110, Jordan
[3]Mathematics Department, Khalifa University of Science and Technology, Abu Dhabi 127788, United Arab Emirates
[4]Mathematics Department, Yarmouk University, Irbid 21163, Jordan
[5]ECSE Department, Case Western Reserve University, Cleveland, OH 44106, USA

Corresponding authors: Mohammad Al-Khaleel (mohammad.alkhaleel@ku.ac.ae) and Fadi Nessir Zghoul (frnessirzghoul@just.edu.jo)

The work of Osama Al-Khaleel was supported by the AMD-XILINX University Program to Jordan University of Science and Technology (JUST).

ABSTRACT Decimal arithmetic is gaining more attention by researchers due to their importance in many human-centric applications. Hardware implementations of decimal arithmetic are preferable over their software counterparts as the former leads in performance especially when it comes to real-time applications. Decimal addition is a main decimal operation as most of the other decimal operation can be implemented using decimal addition. Complementary Metal Oxide Semiconductor (CMOS) technology has the advantage of reducing the average power consumption and the propagation delay in digital systems. Decimal adders can be designed as a ripple carry addition (RCA) structure or as a carry look-ahead addition (CLA) structure with the latter being faster with extra hardware cost. This work proposes two CMOS-based carry look-ahead decimal adders (CLDAs) that can be used within any decimal arithmetic unit. Operands of 1-digit size, 2-digit size, 3-digit size, and 4-digit size are investigated. The Boolean logic for each of the proposed CLDAs is developed and the CMOS realization is provided. The circuit of each of the proposed CLDAs is simulated with 45 *nm* technology library using the LT-SPICE spice simulation tool. The simulation shows promising results in terms of speed, power, and power delay product (PDP).

INDEX TERMS BCD adders, CLDA, CMOS technology, decimal arithmetic, decimal hardware, LT-SPICE, PDP.

I. INTRODUCTION

Binary arithmetic is the core of arithmetic in almost all digital systems [1], [2]. However, decimal arithmetic is opening its way to the digital systems world and gaining more and more attentions by many researchers. This is due to the emerging needs for decimal arithmetic in many human-centric applications including commercial, financial, currency conversion, accounting, tax calculations, banking, database, and Internet-based applications [2], [3], [4], [5], [6], [7].

The associate editor coordinating the review of this manuscript and approving it for publication was Gian Domenico Licciardo.

One approach for such applications to handle decimal arithmetic is to convert from decimal to binary such that the computations are performed using the underlying binary hardware that produces binary result, which is converted back to decimal. However, the conversion process introduces errors due to the inexact conversions between the two number systems. For example, the decimal numbers $(0.7)_{10}$ and $(0.2)_{10}$ do not convert to an exact equivalent binary number and a rounding is imposed. The final results in many cases are significantly affected by the accumulation of these errors and a massive losses might happen [4], [6], [8].

Another approach for these applications to overcome this dilemma and eliminate the conversion errors is to

store the data in decimal format and perform the operations using decimal arithmetic software [3], [4]. However, processing the computations over dedicated hardware is much faster and more powerful than the software libraries solutions [3], [8], [9]. This has more impact with the fact that todays human-centric applications require to process massive amount of data within an acceptable time duration [5].

The importance of decimal arithmetic leads to have the decimal (radix-10) floating-point numbers included in the IEEE 754-2008 standard [10] and IEEE P754/D2.50 [11] as an extension to the original 1985 binary standard [12]. Moreover, the advances in VLSI technology and simulation enhances the speed and power efficiency of hardware units [8], [13]. This makes it feasible to have different commercial processors furnished with dedicated decimal arithmetic units to fulfil the needs of supporting decimal arithmetic in the decimal data based applications. Examples of such commercial processors are the IBM eServer z900 [14], the IBM POWER6 [15], the IBM z10 [16], the IBM zEnterprise-196 [17], and the Fujitsu Sparc64 X [18].

Addition is a key operation in all computational systems as it can be the basis for other arithmetic operations such as subtraction, multiplication, and division. Therefore, the performance of the adder is of high impact on the performance of arithmetic unit and hence the overall performance of any processor.

In decimal arithmetic, binary coded decimal (BCD) is a 4-bit binary code representation for the decimal digits in the decimal number system. Each decimal digit is represented by 4-bit using its binary equivalent. For example, $(2)_{10} = (10)_2$ is represented by $(0010)_{BCD}$ in BCD. Other examples are $(9)_{10} = (1001)_2 = (0010)_{BCD}$ and $(5)_{10} = (101)_2 = (0101)_{BCD}$.

The main advantage of using the BCD representation is to simplify the digital logic design of any decimal arithmetic unit such as the decimal adder or the BCD adder.

A 1-digit BCD adder is a digital circuit that adds two decimal digits, represented in BCD, with a 1-bit carry input and outputs the result as a decimal digit also in BCD format along with a 1-bit carry out.

Since there are only 10 decimal digits in the decimal number system with each digit is represented by 4 bits in the BCD format, the traditional BCD addition of two decimal digits involves a correction whenever the results exceeds the digit $(9)_{10} = (1001)_{BCD}$. This done by adding $(6)_{10} = (0110)_{BCD}$ to the results. Therefore, a traditional BCD adder can be designed using two level of binary addition. In the top-level, the two BCD digits are added with the carry in using a 4-bit binary adder. Whereas, in the bottom level, $(6)_{10} = (0110)_{BCD}$ is added, using a second 4-bit binary adder, with the result from the top level whenever a correction is needed. A traditional 1-digit BCD adder that adds the two BCD digits $A = (A_3A_2A_1A_0)_{BCD}$ and $B = (B_3B_2B_1B_0)_{BCD}$ with the carry in C_{in} and produces the BCD results $S = (S_3S_2S_1S_0)_{BCD}$ along with the carry out C_{out} is illustrated in Figure 1.

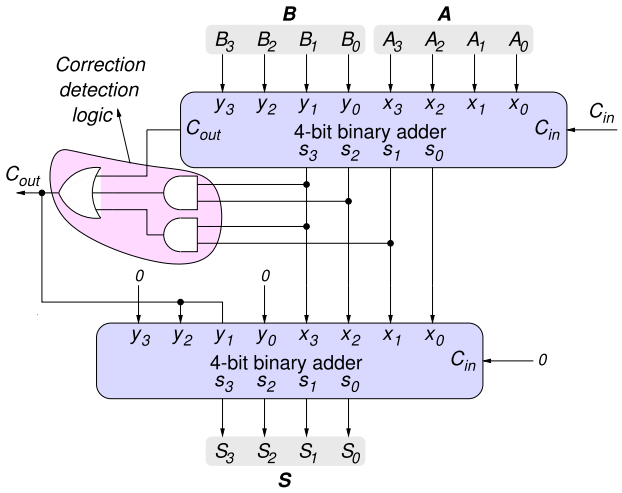


FIGURE 1. Traditional 1-digit BCD adder.

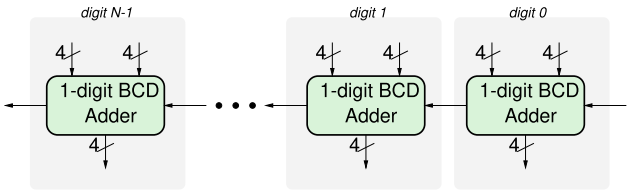


FIGURE 2. *N*-digit Ripple Carry Decimal Adder (RCDA).

An N-digit BCD adder can be designed by cascading N 1-digit BCD adders in a ripple carry fashion as shown in Figure 2. This work refers to this adder as ripple carry decimal adder (RCDA).

In the binary adders, the ripple carry addition is known to be the slowest addition scheme. The propagation delay of the carry out of the 1-digit BCD adder is relatively higher than that of a 4-bit binary adder since it must passes through the correction detection logic as shown in Figure 1. Such delay would have more impact on the speed when it comes to the N-digit RCDA since the carry signal must traverse all of the N 1-digit BCD adders. Therefore, the RCDA suffers a speed problem.

In binary adders, carry look ahead addition (CLA) is one approach that is used to break down the carry propagation in the ripple carry addition (RCA). In CLA the carry out from each stage, which is input to the next stage, is computed in terms of the inputs such that it no longer depends on the carry out from the previous stage. This means that there is no carry propagation and the ripple carry chain is broken. CLA addition can be utilized in a similar way in order to speed up the decimal addition by breaking down the carry chain in the RCDA. This work refers to the BCD adder that utiles the CLA approach as carry look ahead decimal adder (CLDA).

This work proposes two CMOS-based carry look-ahead BCD adders: Carry look-ahead decimal adder 1 ($CLDA_1$) and carry look-ahead decimal adder 2 ($CLDA_2$). The LTSPICE spice simulation tool with 45nm technology library is used

to simulate the proposed designs. This work also derives the Boolean equations and provides the transistor level schematic for each of the proposed BCD adder.

While this work focuses on CMOS technology for its widespread adoption and accessibility, it should be mentioned that advanced technologies such as FinFET and CNT devices can be investigated with the proposed architectures for possibly further enhancement on the performance.

The rest of this document is structured as follows: the related work and background are covered in Sections II and III, respectively. The proposed carry look-ahead decimal adders are presented in Section IV. The experimental results, the comparisons, and the discussions are presented in Section V. Finally, the conclusions are provided in Section VI.

II. RELATED WORK

Due to its importance, decimal arithmetic operations and their implementations are attractive to wide range of research works all over the world. Decimal addition is consider the basis of all other decimal operations. This is due to the fact that all other decimal operation can be performed utilizing decimal addition. Decimal addition is implemented in software as a soft application or in hardware as a physical decimal adder, with the latter being much faster. In both cases, two or multi-operand decimal addition can be of interest depending on the requirements by the application.

An early work where fast and economical BCD adders are investigated at the gate level is presented in [19]. Another gate level implementation is found in [20], where an addition/subtraction module is proposed. Many more works related to decimal arithmetic are there in the literature such as those presented by [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], and [43].

Despite the existence of many works in the literature that address the hardware implementations of the decimal addition, most of these works are high level implementations. Unfortunately, only few low-level (transistor-level) implementations do exist. The work in [44] proposes a 4-digit BCD adder using the dynamic-threshold voltage MOSFET (DTMOS) technique. A lower supply voltage is used for low leakage current and low power dissipation. Three different CMOS technologies are used with the Tanner EDA Tool to simulate the design. These CMOS technologies are 22 *nm*, 32 *nm*, and 45 *nm*.

The authors of [45] propose an 8-digit BCD adder using different techniques to reduce the power consumption and the leakage power. These techniques are the dual voltage technology (DVT), the power gating technique, and the gate diffusion input (GDI). The design is simulated using the Tanner EDA Tool with 130 *nm* technology. A novel CMOS 1-digit BCD adder correction circuit is proposed in [46]. The goal is to achieve high speed and power efficiency by replacing the complex full adders logic at the output stage by a 2×1 multiplexers that are based on pass transistor logic (PTL). The proposed strategy optimizes the critical path delay for the carry signal, reducing the power-delay product (PDP) comparing to conventional strategies. The work is implemented using 32*nm* CMOS technology through T-Spice simulations. An 8-digit power gated BCD adder is proposed by [47]. Two different full adder circuits are investigated when building the BCD adder. These are 28T full adder circuit and 14T full adder circuit. The Tanner EDA and TSMC_180 *nm* technology are used to simulate the full adders and the BCD adder and a significant power reduction is achieved.

The Clock Gated Power Gating and the DVT (Dual-vth) techniques are used by [48] to design a low-voltage and low-power 1-digit BCD adder. The DVT scheme is used in designing the full adder block such that the leakage power is reduced while maintaining high performance of top level circuit, which is simulated using 45 *nm* technology and multiple runs on spice.

The work in [49] proposes 1-digit BCD adder. The goal is to achieve a compact and high speed BCD addition circuit. Three different full adder circuits are investigated against the 28T conventional full adder circuit. These full adder circuits are the 10T, the 8T, and the 6T. The simulation is carried out using the Cadence Virtuoso CAD tool with 180 *nm* and 90 *nm* technologies. The 6T full adder circuit shows the best area, power, and PDP when using designing the 1-digit BCD adder.

The authors of [50] present the designs of one-digit, two-digit, three-digit, and four-digit decimal adders that are implemented in CMOS technology. The circuits of these designs are simulated using the LT-SPICE spice simulator software with 45 *nm*, 65 *nm*, and 180 *nm* technologies. Following the traditional BCD addition circuitry, the work also designs five different BCD adders using five different binary carry look-ahead adders (CLAs) that exist in the literature. One of these five binary CLAs is the conventional CLA, which is addressed by many works in the literature such as [51], [52], [53], [54], [55], [56], [57], and [58]. The other four binary CLAs are the modified 4-bit CLA architecture of [59], which is an improvement of the design in [60], the Compact 4-bit CLA architecture of [61], the Novel 4-bit CLA architecture presented in [62], and the high-speed 4-bit CLA architecture of [53].

To the best of the authors' knowledge, there is no work in the literature that addresses the design of BCD adder based on the carry look-ahead addition at the transistor level. Therefore, this work presents the design of BCD adder circuits utilizing the carry look-ahead technique. The ripple carry model presented in [50] is used as the seed to build the carry look-ahead model for the CLA bcd adders presented in this work. Two different carry look-ahead models are developed and implemented in CMOS using the 45 *nm* technology library.

III. BACKGROUND

The work in [50] proposes the two-netlist CMOS based BCD 1-digit adder shown in Figure 3. In developing the

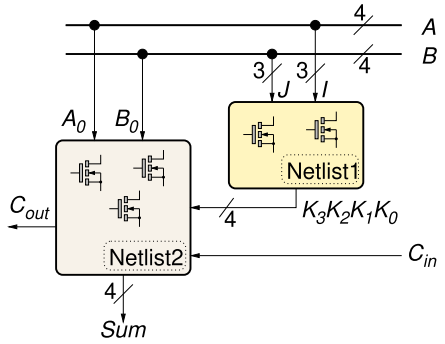


FIGURE 3. The 1-digit BCD adder of [50].

mathematical model for their design, the authors assume that the decimal inputs of the BCD adder are the two operands A and B, while, the decimal carry input is C_{in}. $A = (A_3A_2A_1A_0)_{BCD}$ and $B = (B_3B_2B_1B_0)_{BCD}$, where $A, B \in \{0, 9\}$. The output decimal sum $S \in \{0, 9\}$ is assumed to be $S = (S_3S_2S_1S_0)_{BCD}$ and the output decimal carry be C_{out}.

The outputs S_3, S_2, S_1, S_0, and C_{out} are produced by the lower netlist (Netlist2). The Boolean equations that are derived for S_3, S_2, S_1, S_0, and C_{out} are given by (1).

$$S_0 = \overline{\overline{Y_0} \cdot \overline{Y_1} \cdot \overline{Y_2} \cdot \overline{Y_3}}, \tag{1a}$$
$$S_1 = \overline{\overline{Y_4} \cdot \overline{Y_5} \cdot \overline{Y_6} \cdot \overline{Y_7} \cdot \overline{Y_8} \cdot \overline{Y_9}}, \tag{1b}$$
$$S_2 = \overline{\overline{Y_{10}} \cdot \overline{Y_{11}} \cdot \overline{Y_{12}} \cdot \overline{Y_{13}} \cdot \overline{Y_{14}} \cdot \overline{Y_{15}} \cdot \overline{Y_{16}}}, \tag{1c}$$
$$S_3 = \overline{\overline{Y_{17}} \cdot \overline{Y_{18}} \cdot \overline{Y_{19}} \cdot \overline{Y_{20}} \cdot \overline{Y_{21}} \cdot \overline{Y_{22}}}, \tag{1d}$$
$$C_{out} = \overline{\overline{Y_{23}} \cdot \overline{Y_{24}} \cdot \overline{Y_{25}} \cdot \overline{Y_{26}}}, \tag{1e}$$

where

$$\begin{aligned}
Y_0 &:= C_{in}\bar{A}_0\bar{B}_0, & Y_1 &:= \bar{C}_{in}A_0\bar{B}_0,\\
Y_2 &:= \bar{C}_{in}\bar{A}_0B_0, & Y_3 &:= C_{in}A_0B_0,\\
Y_4 &:= \bar{K}_2\bar{K}_0C_{in}B_0, & Y_5 &:= \bar{K}_2\bar{K}_0C_{in}A_0,\\
Y_6 &:= \bar{K}_2\bar{K}_0A_0B_0, & Y_7 &:= K_0\bar{A}_0\bar{B}_0,\\
Y_8 &:= K_0\bar{C}_{in}\bar{B}_0, & Y_9 &:= K_0\bar{C}_{in}\bar{A}_0,\\
Y_{10} &:= \bar{K}_1K_0C_{in}B_0, & Y_{11} &:= \bar{K}_1K_0C_{in}A_0,\\
Y_{12} &:= \bar{K}_1K_0A_0B_0, & Y_{13} &:= K_1\bar{C}_{in}\bar{B}_0,\\
Y_{14} &:= K_1\bar{C}_{in}\bar{A}_0, & Y_{15} &:= K_1\bar{A}_0\bar{B}_0,\\
Y_{16} &:= K_1\bar{K}_0, & Y_{17} &:= K_1K_0C_{in}B_0,\\
Y_{18} &:= K_1K_0C_{in}A_0, & Y_{19} &:= K_1K_0A_0B_0,\\
Y_{20} &:= K_2\bar{C}_{in}\bar{B}_0, & Y_{21} &:= K_2\bar{C}_{in}\bar{A}_0,\\
Y_{22} &:= K_2\bar{A}_0\bar{B}_0, & Y_{23} &:= K_2C_{in}B_0,\\
Y_{24} &:= K_2C_{in}A_0, & Y_{25} &:= K_2A_0B_0,\\
Y_{26} &:= K_3.
\end{aligned}$$

K_3, K_2, K_1, and K_0 are generated by the upper netlist (Netlist1) and are specified by the Boolean equations as given by (2).

$$K_0 = \overline{\overline{X_0} \cdot \overline{X_1} \cdot \overline{X_2} \cdot \overline{X_3} \cdot \overline{X_4} \cdot \overline{X_5} \cdot \overline{X_6} \cdot \overline{X_7}}, \tag{2a}$$
$$K_1 = \overline{\overline{X_0} \cdot \overline{X_8} \cdot \overline{X_9} \cdot \overline{X_{10}} \cdot \overline{X_{11}} \cdot \overline{X_{12}} \cdot \overline{X_{13}} \cdot \overline{X_7}}, \tag{2b}$$
$$K_2 = \overline{\overline{X_{14}} \cdot \overline{X_{15}} \cdot \overline{X_{16}} \cdot \overline{X_{17}} \cdot \overline{X_{18}}}, \tag{2c}$$
$$K_3 = \overline{\overline{X_{19}} \cdot \overline{X_{20}} \cdot \overline{X_{21}} \cdot (\overline{X_{22}} \cdot \overline{X_{23}}) \cdot \overline{X_{24}}) \cdot \overline{X_7}}, \tag{2d}$$

where

$$\begin{aligned}
X_0 &:= \bar{A}_3\bar{A}_2\bar{A}_1B_2B_1, & X_1 &:= A_1\bar{B}_3\bar{B}_2\bar{B}_1,\\
X_2 &:= \bar{A}_3\bar{A}_1\bar{B}_2B_1, & X_3 &:= \bar{A}_2A_1B_2\bar{B}_1,\\
X_4 &:= A_2A_1B_2B_1, & X_5 &:= A_2\bar{A}_1B_3,\\
X_6 &:= A_3B_2\bar{B}_1, & X_7 &:= A_3B_3,\\
X_8 &:= A_2\bar{B}_3\bar{B}_2\bar{B}_1, & X_9 &:= \bar{A}_3\bar{A}_2B_2\bar{B}_1,\\
X_{10} &:= A_2\bar{A}_1\bar{B}_2B_1, & X_{11} &:= \bar{A}_2A_1\bar{B}_2B_1,\\
X_{12} &:= A_2A_1B_3, & X_{13} &:= A_3B_2B_1,\\
X_{14} &:= \bar{A}_3\bar{A}_2\bar{A}_1B_3, & X_{15} &:= A_3\bar{B}_3\bar{B}_2\bar{B}_1,\\
X_{16} &:= A_2\bar{A}_1B_2\bar{B}_1, & X_{17} &:= A_2A_1\bar{B}_2B_1,\\
X_{18} &:= \bar{A}_2A_1B_2B_1, & X_{19} &:= A_2A_1B_2,\\
X_{20} &:= A_2\bar{A}_1B_3, & X_{21} &:= A_2B_2B_1,\\
X_{22} &:= A_3B_2\bar{B}_1, & X_{23} &:= A_1B_3,\\
X_{24} &:= A_3B_1.
\end{aligned}$$

IV. THE PROPOSED CARRY LOOK AHEAD DECIMAL ADDERS (CLDAS)

A N-digit BCD adder can be build by cascading N adders of 1-digit BCD adder shown in Figure 3 [50]. Figure 4 illustrates this process for $N = 4$. It can be easily figured out that the carry rippling slows down the speed of the BCD addition process; especially for large N.

One efficient solution to such dilemma is the utilization of a carry look-ahead technique such that the carry propagation in the carry chain is no longer a function of any previous carry but only a function of the inputs themselves. To achieve this, this work moves out the computation of the carry outputs, which is performed in Netlist2 of Figure 3 [50], to a dedicated carry look-ahead logic units as demonstrated by Figure 5. The sum outputs are still computed in the same way as in [50]. However, each carry output in the proposed design is computed using its own CLA logic unit.

By refereeing to Equation (1e), it can be generalized that the carry out C_{i+1} from the i_{th} 1-digit BCD adder in a N-digit RCDA adder is computed according to (3).

$$C_{i+1} = \overline{\overline{K_{4i+2}C_iB_{4i}} \cdot \overline{K_{4i+2}C_iA_{4i}} \cdot \overline{K_{4i+2}A_{4i}B_{4i}} \cdot \overline{K_{4i+3}}} \tag{3}$$

It is clear that C_{i+1} is a function of C_i and hence the rippling is there. The carry look ahead addition breaks the carry chain by substituting a lower carry out into the upper carry out such that any carry is a function of the inputs only. In the case of this work, the 4 carry outputs for the 4-digit BCD adder (C_1, C_2, C_3, and C_4) are computed according to (3) as shown

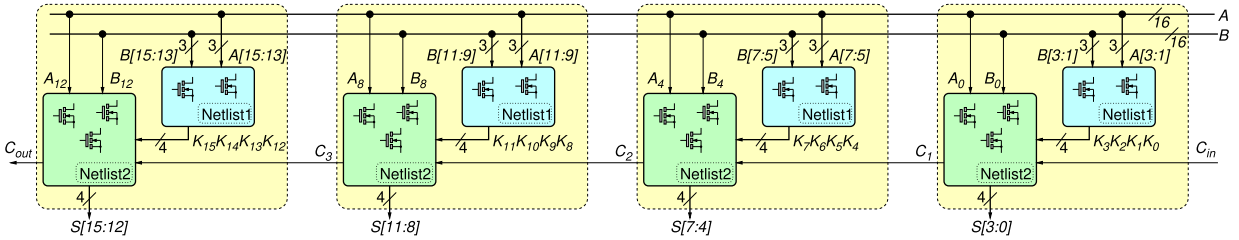


FIGURE 4. 4-digit Ripple Carry Decimal Adder (RCDA) [50].

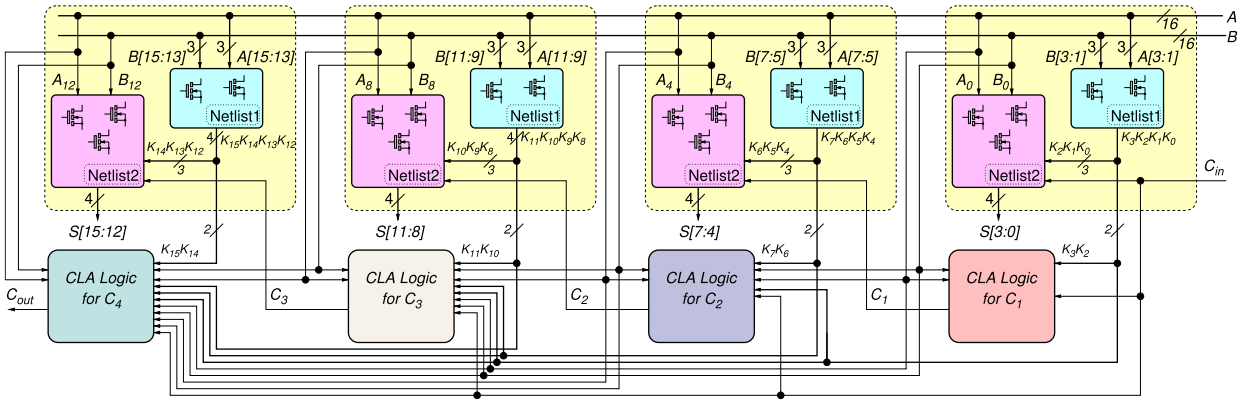


FIGURE 5. A block diagram of the proposed 4-digit CLDA.

by (4), (5), (6), (7).

$$C_1 = \overline{\overline{K_2C_0B_0} \cdot \overline{K_2C_0A_0} \cdot \overline{K_2A_0B_0} \cdot \overline{K_3}}, \tag{4}$$

$$C_2 = \overline{\overline{K_6C_1B_4} \cdot \overline{K_6C_1A_4} \cdot \overline{K_6A_4B_4} \cdot \overline{K_7}}, \tag{5}$$

$$C_3 = \overline{\overline{K_{10}C_2B_8} \cdot \overline{K_{10}C_2A_8} \cdot \overline{K_{10}A_8B_8} \cdot \overline{K_{11}}}, \tag{6}$$

$$C_4 = \overline{\overline{K_{14}C_3B_{12}} \cdot \overline{K_{14}C_3A_{12}} \cdot \overline{K_{14}A_{12}B_{12}} \cdot \overline{K_{15}}}. \tag{7}$$

Given that any K_i is a function of the inputs only and that $C_0 = C_{in}$ in (4), substituting (4) into (5), one gets C_2 in terms of the inputs and it is no more depending on C_1. The resulted C_2 is substituted into (6) to obtain a formula for C_3 that depends on the inputs only. A formula for C_4 that depends only on the inputs and independent from any previous carry can be derived in a similar way.

Based on this, this work proposes two different designs for the CLA units in Figure 5, and hence two decimal adders are proposed and implemented using CMOS technology. These two proposed decimal adders are: $CLDA_1$ and $CLDA_2$. For $CLDA_1$, the Boolean logic for the carry outputs is presented by (8), (9), (10), and (11) for C_1, C_2, C_3, and C_4, respectively.

$$C_1 = \overline{\bar{H}_0.\bar{K}_3}, \tag{8}$$

$$C_2 = \overline{\bar{L}_0.\bar{L}_1.\bar{H}_1}, \tag{9}$$

$$C_3 = \overline{\bar{M}_0.\bar{M}_1.\bar{L}_2.\bar{H}_2}, \tag{10}$$

$$C_4 = \overline{\bar{N}_0.\bar{N}_1.\bar{M}_2.\bar{L}_3.\bar{H}_3}, \tag{11}$$

where

$$H_0 = \overline{\overline{Z_0}.\overline{Z_1}.\overline{Z_2}}, \quad H_1 = \overline{\overline{Z_7}.\overline{K_7}}, \quad H_2 = \overline{\overline{Z_{14}}.\overline{K_{11}}},$$
$$H_3 = \overline{\overline{Z_{23}}.\overline{K_{15}}}, \quad L_0 = \overline{\overline{Z_3}.\overline{Z_4}}, \quad L_1 = \overline{\overline{Z_5}.\overline{Z_6}},$$
$$L_2 = \overline{\overline{Z_{12}}.\overline{Z_{13}}}, \quad L_3 = \overline{\overline{Z_{21}}.\overline{Z_{22}}}, \quad M_0 = \overline{\overline{Z_8}.\overline{Z_9}},$$
$$M_1 = \overline{\overline{Z_{10}}.\overline{Z_{11}}}, \quad M_2 = \overline{\overline{Z_{19}}.\overline{Z_{20}}}, \quad N_0 = \overline{\overline{Z_{15}}.\overline{Z_{16}}},$$
$$N_1 = \overline{\overline{Z_{17}}.\overline{Z_{18}}},$$

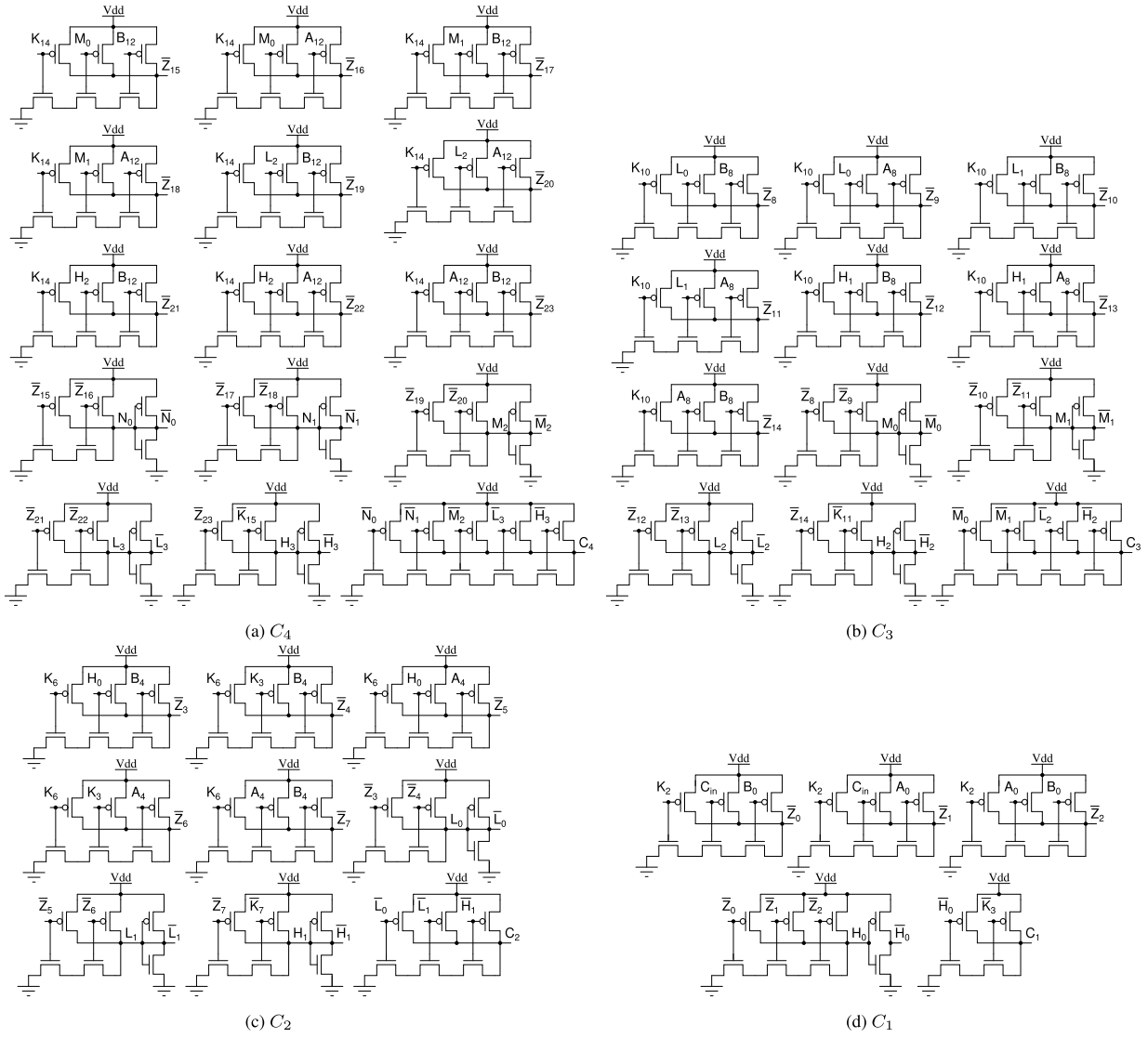
and

$$Z_0 := K_2C_{in}B_0, \quad Z_1 := K_2C_{in}A_0,$$
$$Z_2 := K_2A_0B_0, \quad Z_3 := K_6H_0B_4,$$
$$Z_4 := K_6K_3B_4, \quad Z_5 := K_6H_0A_4,$$
$$Z_6 := K_6K_3A_4, \quad Z_7 := K_6A_4B_4,$$
$$Z_8 := K_{10}L_0B_8, \quad Z_9 := K_{10}L_0A_8,$$
$$Z_{10} := K_{10}L_1B_8, \quad Z_{11} := K_{10}L_1A_8,$$
$$Z_{12} := K_{10}H_1B_8, \quad Z_{13} := K_{10}H_1A_8,$$
$$Z_{14} := K_{10}A_8B_8, \quad Z_{15} := K_{14}M_0B_{12},$$
$$Z_{16} := K_{14}M_0A_{12}, \quad Z_{17} := K_{14}M_1B_{12},$$
$$Z_{18} := K_{14}M_1A_{12}, \quad Z_{19} := K_{14}L_2B_{12},$$
$$Z_{20} := K_{14}L_2A_{12}, \quad Z_{21} := K_{14}H_2B_{12},$$
$$Z_{22} := K_{14}H_2A_{12}, \quad Z_{23} := K_{14}A_{12}B_{12}.$$

The CMOS realization for the *CLA* logic in Figure 5 is presented by Figure 6 (for the proposed $CLDA_1$).

On the other hand, for $CLDA_2$, the Boolean logic for the carry outputs is presented by (12), (13), (14), and (15) for C_1, C_2, C_3, and C_4, respectively. The CMOS realization for the *CLA* logic in Figure 5 is presented by Figure 7 (for the proposed $CLDA_2$).

The CMOS realizations presented in Figures 6 and 7 are directly derived from the corresponding Boolean logic



(a) C_4 (b) C_3 (c) C_2 (d) C_1

FIGURE 6. Carry signals in $CLDA_1$ design.

equations without any optimization. This approach ensures transparency in design and facilitates a straightforward understanding of the proposed circuits, while leaving scope for potential enhancements through advanced low-level optimization techniques in future research.

$$C_1 = \overline{\bar{H}_0.\bar{K}_3}, \tag{12}$$

$$C_2 = \overline{\bar{L}_0.\bar{L}_1.\bar{H}_1}, \tag{13}$$

$$C_3 = \overline{\bar{M}_0.\bar{M}_1.\bar{L}_2.\bar{H}_2}, \tag{14}$$

$$C_4 = \overline{\bar{N}_0.\bar{N}_1.\bar{M}_2.\bar{L}_3.\bar{H}_3}, \tag{15}$$

where

$$\begin{aligned}
&H_0 = \overline{\overline{P_0}.\overline{P_1}.\overline{P_2}},\ H_1 = \overline{\overline{P_5}.\overline{K_7}},\ H_2 = \overline{\overline{P_6}.\overline{K_{11}}},\\
&H_3 = \overline{\overline{P_7}.\overline{K_{15}}},\quad L_0 = X_0.B_4,\ L_1 = X_0.A_4,\\
&L_2 = X_1.Y_4,\quad L_3 = X_2.Y_5,\ M_0 = X_3.Y_4,\\
&M_1 = X_4.Y_4,\quad M_2 = X_5.Y_5,\ N_0 = X_6.Y_5,\\
&N_1 = X_7.Y_5,
\end{aligned}$$

and

$$\begin{aligned}
&P_0 := K_2C_{in}B_0,\ P_1 := K_2C_{in}A_0,\\
&P_2 := Y_0K_2,\quad P_3 := K_6H_0,\\
&P_4 := K_6K_3,\quad P_5 := Y_1K_6,\\
&P_6 := Y_2K_{10},\quad P_7 := Y_3K_{14},\\
&X_0 := \overline{\overline{P_3}.\overline{P_4}},\quad X_1 := K_{10}.H_1,\\
&X_2 := K_{14}.H_2\quad X_3 := K_{10}.L_0,\\
&X_4 := K_{10}.L_1,\quad X_5 := K_{14}.L_2,\\
&X_6 := K_{14}.M_0,\quad X_7 := K_{14}.M_1\\
&Y_0 := A_0.B_0\quad Y_1 := A_4.B_4,\\
&Y_2 := A_8.B_8,\quad Y_3 := A_{12}B_{12},\\
&Y_4 := \overline{\bar{A}_8.\bar{B}_8},\quad Y_5 := \overline{\bar{A}_{12}.\bar{B}_{12}}.
\end{aligned}$$

It should be pointed out that the carry look-ahead addition (CLA) schemes are usually built for operands with limited size. This is due to the fact that, in such schemes, the costly extra hardware penalty imposed on the overall design grows up with the size of the operands. As a result, building carry

look-ahead adders with large operands may exaggerate the overall cost of the design. In addition, the fan-in of the gates also grows with the size of the operands [63]. In CMOS technology, a gate with fan-in much grater than 4 can cause poor noise immunity and poor rise and fall time, which results in poor propagation delay. Therefore, practically, binary CLAs are usually designed for operand sizes of up to 4-bit. Higher operand sizes are served by cascading multiple of the 4-bit binary carry look-ahead adder in a ripple carry fashion. Considering these facts, in this work, both of the proposed N-digit CLDAs are build for up to $N = 4$. In fact, 1-digit 2-digit 3-digit and 4-digit CLDAs are investigated so that a combination of these individual components can be used to build higher order decimal adder. For example, a 5-digit BCD adder can be built by cascading one 4-digit CLDA with a 1-digit CLDA or by cascading one 2-digit CLDA with one 3-digit CLDA. Another example is an 8-digit BCD adder, which can be built by cascading four 2-digit CLDAs or by cascading two 4-digit CLDA or by cascading one 2-digit CLDA with two 3-digit CLDAs; and so on.

While our focus in this work is primarily on the CMOS transistor-level design and its corresponding logic implementation, it should be pointed out that the layout of the proposed CLDAs can be structured to ensure a compact and regular design. This layout approach minimizes interconnect lengths and associated parasitic capacitances, which are critical for achieving high-speed performance. Symmetrical structures can be adopted wherever it is possible to maintain balanced signal propagation paths, thereby minimizing timing mismatches and improving overall circuit reliability. Using the Predictive Technology Model (PTM) for the 45 *nm* technology node, the effects of parasitic capacitances and resistances are carefully modeled and analyzed during the simulation process. These parasitics, inherent in physical layouts, can introduce significant delay and power overheads. By incorporating these factors into the simulations, the proposed designs are optimized to balance speed and power consumption while maintaining high reliability. The choice of the 45 *nm* technology node is driven by its suitability for modern fabrication processes and its ability to deliver an optimal trade-off between power efficiency and switching speed. The transistor dimensions and parameters adhered to the PTM specifications, ensuring the proposed designs can be manufactured under contemporary CMOS technology.

V. RESULTS

The Boolean logic of each of the proposed CLDAs is functionally tested and verified using the simulation tools from AMD-Xilinx. A block diagram of the elaborated design for the proposed $BCDA_g$ is shown by Figure 8.

The proposed designs are simulated, for $N = 1$, $N = 2$, $N = 3$, and $N = 4$, using the 45 *nm* Predictive Technology Model (PTM) BSIM4 library (developed at the University of Minnesota) with a supply voltage of 1V, typical process conditions, and room temperature (27°C). These PTM models are evolved from the earlier Berkeley Predictive Technology Model by the Device Group at the University of California, Berkeley. All simulations are conducted employing this PTM 45 *nm* library to ensure an accurate representation of the CMOS device characteristics. The LT-SPICE free SPICE simulator software is used for the simulation. Transient and DC analyses are performed to evaluate the power consumption, propagation delay, and power-delay product (PDP) across the different designs. A clock frequency of 100 MHz is applied for the transient analysis to evaluate the speed and performance metrics under realistic operating conditions. The width of the inverter is set to 135 *nm* for the *NMOS* transistor and 270 *nm* for the *PMOS* transistor.

Based on the traditional BCD 1-digit adder, which is shown in Figure 1, the work in [50] builds five different BCD adders for comparison purposes. In these five BCD adders low-level binary adder designs from [53], [60], [61], and [62] are used. This work compares the proposed decimal adders against these five adders, the decimal adder of [50], and the traditional decimal adder that is based on the conventional binary adder. The symbols presented in Table 1 are used for easiness and clarity.

TABLE 1. Symbols used to refer to the proposed BCD adders and other BCD adders from the literature.

BCD Adder	Symbol
BCD adder using the conventional binary adder	$BCDA_a$
BCD adder using the binary adder in [60]	$BCDA_b$
BCD adder using the binary adder in [61]	$BCDA_c$
BCD adder using the binary adder in [62]	$BCDA_d$
BCD adder using the binary adder in [53]	$BCDA_e$
BCD adder of [50]	$BCDA_f$
Proposed $CLDA_1$ of this work	$BCDA_g$
Proposed $CLDA_2$ of this work	$BCDA_h$

In order to experimentally find the critical path delay in different 1-digit, 2-digit, 3-digit, and 4-digit BCD adders, the delay of all candidate paths must be investigated. Therefore, in this work, the delay of all candidate paths from the inputs to the last carry and the delay of all candidate paths from the inputs to the MSB in the decimal Sum output are experimentally found and reported in Tables 2, 3, 4, and 5, respectively. For example, Table 5 reports these path delays for the proposed 4-digit $BCDA_g$, the proposed 4-digit $BCDA_h$, and the other 4-digit BCD adders from the literature. In this scenario, the maximum delay from the inputs to S_{15} is shown to be 702.41ps, 756.67ps, 759.00ps, 812.00ps, 1025.18ps, 467.38ps, 395.19ps and 396.54ps for the 4-digit $BCDA_a$, 4-digit $BCDA_b$, 4-digit $BCDA_c$, 4-digit $BCDA_d$, 4-digit $BCDA_e$, the 4-digit $BCDA_f$, the proposed 4-digit $BCDA_g$, and the proposed 4-digit $BCDA_h$, respectively.

Tables 2, 3, 4, and 5 generally show that each of the proposed decimal adders outperforms all other decimal adders in terms of speed, especially for larger operands. For example, as shown by Table 2, both of the proposed BCD adders outperform the other decimal adders in terms of speed for 1-digit operands excluding the $BCDA_f$, which

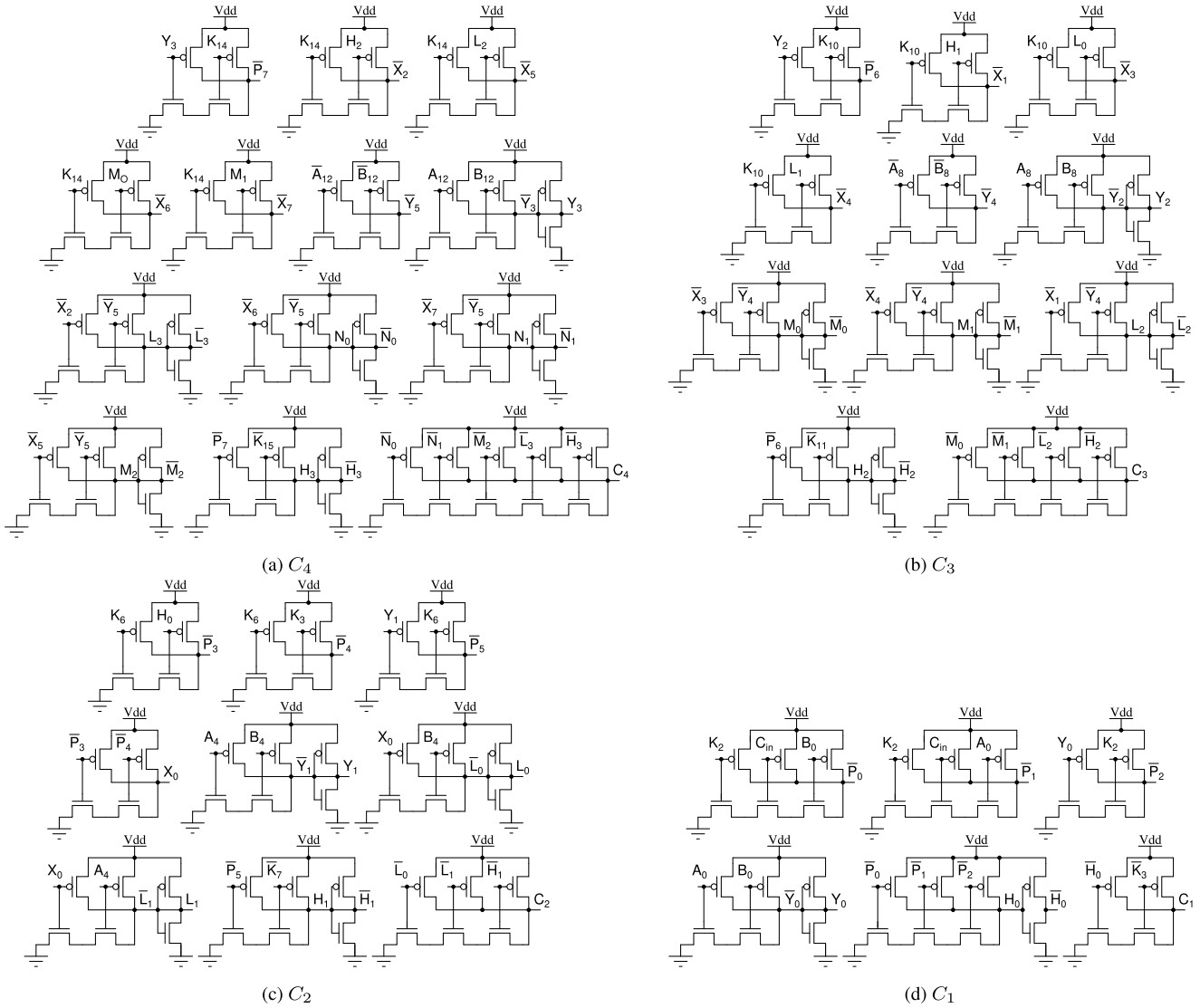


FIGURE 7. Carry signals in ***CLDA*$_2$** design.

shows similar performance. With regard to 2-digit operands (Table 3), the proposed $BCDA_g$ outperforms $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, and $BCDA_e$, with a speedup of almost 1.61, 1.49, 1.50, 1.59, and 1.95, respectively. While the proposed $BCDA_h$ outperforms $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$ and $BCDA_f$, with a speedup of almost 1.69, 1.56, 1.58, 1.67, 2.05, and 1.04, respectively.

For 3-digit operands, as can be figured out from Table 4, the proposed $BCDA_g$ shows a speed up of 1.84, 1.73, 1.74, 1.86, 2.33, and 1.10 comparing to $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$ and $BCDA_f$, respectively. While, the proposed $BCDA_h$ shows a speed up of 1.87, 1.77, 1.78, 1.90, 2.37, and 1.12 comparing to $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$ and $BCDA_f$, respectively.

Additionally, for 4-digit operands, Table 5 shows that the proposed $BCDA_g$ is 1.80, 1.91, 1.92, 2.05, 2.59, and 1.18 times faster than $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively. While, the proposed $BCDA_h$ decimal adder is 1.79, 1.91, 1.91, 2.05, 2.59, and 1.18 times faster than $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively. It can be noted that as the operand size increases, both of the proposed BCD adders achieve greater speed improvements, with the proposed $BCDA_g$ showing a higher enhancement ratio.

Moreover, the proposed $BCDA_g$ and the proposed $BCDA_h$ are compared to $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$ in terms of Average Power, Propagation Delay, Power Delay Product (PDP), and Transistors Count (TC). Tables 6, 7, 8, and 9 list these comparisons.

For operand sizes of 3-digits and above, it is apparent that the proposed BCD adders performs better than all other adders in terms of power-delay product. In all circumstances, the cost is a higher transistor count, and in certain cases, it is also a higher power dissipation. For example, for 4-digit

TABLE 2. Delay of all possible critical paths for 1-digit decimal adders.

Path	Delay(ps)							
	$BCDA_a$	$BCDA_b$ [60]	$BCDA_c$ [61]	$BCDA_d$ [62]	$BCDA_e$ [53]	$BCDA_f$ [50]	$BCDA_g$	$BCDA_h$
C_{in} to S_3	221.79	219.90	227.51	242.33	284.73	74.66	75.16	75.22
a_0 to S_3	233.07	236.99	249.42	222.98	275.60	81.68	81.16	81.58
a_1 to S_3	208.48	219.13	198.10	210.31	247.27	232.19	232.19	232.18
a_2 to S_3	230.02	211.56	177.44	184.59	230.91	172.56	172.55	172.57
a_3 to S_3	146.67	103.06	140.13	151.32	143.51	148.67	148.65	148.65
Max delay to S_3	233.07	236.99	249.42	242.33	284.73	232.19	232.19	232.18
C_{in} to C_{out}	137.70	130.02	143.25	162.16	185.67	43.22	59.74	59.64
a_0 to C_{out}	151.56	148.18	165.67	144.03	179.53	45.55	62.67	77.43
a_1 to C_{out}	127.31	130.43	114.24	130.04	153.37	142.91	160.83	153.23
a_2 to C_{out}	124.73	132.35	116.39	133.21	150.28	99.15	155.66	149.05
a_3 to C_{out}	75.03	108.06	115.50	109.56	165.93	121.27	151.76	144.51
Max delay to C_{out}	151.56	148.18	165.67	162.16	185.67	142.91	160.83	153.23

TABLE 3. Delay of all possible critical paths for 2-digit decimal adders.

Path	Delay(ps)							
	$BCDA_a$	$BCDA_b$ [60]	$BCDA_c$ [61]	$BCDA_d$ [62]	$BCDA_e$ [53]	$BCDA_f$ [50]	$BCDA_g$	$BCDA_h$
C_{in} to S_7	393.49	389.04	397.94	422.88	522.86	175.43	179.31	174.23
a_0 to S_7	407.82	408.06	417.79	406.35	517.48	179.12	180.57	190.06
a_1 to S_7	383.58	389.49	365.10	391.19	490.82	276.61	279.28	266.29
a_2 to S_7	449.22	416.14	419.81	444.10	545.09	271.76	272.87	263.13
a_3 to S_7	392.46	386.08	385.36	411.45	501.77	266.97	267.49	258.46
Max delay to S_7	449.22	416.14	419.81	444.10	545.09	276.61	279.28	266.29
C_{in} to C_{out}	311.90	299.46	312.21	345.59	425.30	136.44	117	120.61
a_0 to C_{out}	323.87	317.96	334.58	327.33	420.03	139.63	117.48	135.99
a_1 to C_{out}	302.59	299.57	281.74	313.80	393.47	237.55	216.28	214.25
a_2 to C_{out}	364.63	326.11	335.33	366.55	447.26	233.21	211	206.65
a_3 to C_{out}	309.57	295.13	301.64	333.73	403.72	227.58	204.04	204.27
Max delay to C_{out}	364.63	326.11	335.33	366.55	447.26	237.55	216.28	214.25

TABLE 4. Delay of all possible critical paths for 3-digit decimal adders.

Path	Delay(ps)							
	$BCDA_a$	$BCDA_b$ [60]	$BCDA_c$ [61]	$BCDA_d$ [62]	$BCDA_e$ [53]	$BCDA_f$ [50]	$BCDA_g$	$BCDA_h$
C_{in} to S_{11}	568.37	557.39	565.74	607.84	764.93	270.81	236.5	236.31
a_0 to S_{11}	579.94	577.01	588.63	588.14	757.98	273.16	236.9	252.45
a_1 to S_{11}	559.32	558.49	534.80	576.01	729.97	371.75	337.34	330.90
a_2 to S_{11}	620.20	584.52	588.19	628.94	785.62	367.06	334.87	323.39
a_3 to S_{11}	565.63	552.86	555.97	595.87	741.28	361.91	327	320.16
Max delay to S_{11}	620.20	584.52	588.63	628.94	785.62	371.75	337.34	330.90
C_{in} to C_{out}	484.07	467.99	481.46	528.45	666.51	231.78	166.84	176.29
a_0 to C_{out}	496.25	488.37	503.02	510.41	660.31	234.07	167.43	189.59
a_1 to C_{out}	474.68	469.83	450.78	496.72	633.06	332.69	268.13	267.18
a_2 to C_{out}	539.30	496.21	504.91	549.71	688.22	328.09	264.77	261.29
a_3 to C_{out}	481.71	464.96	470.06	517.09	643.95	322.49	257.08	258.55
Max delay to C_{out}	539.30	496.21	504.91	549.71	688.22	332.69	268.13	267.18

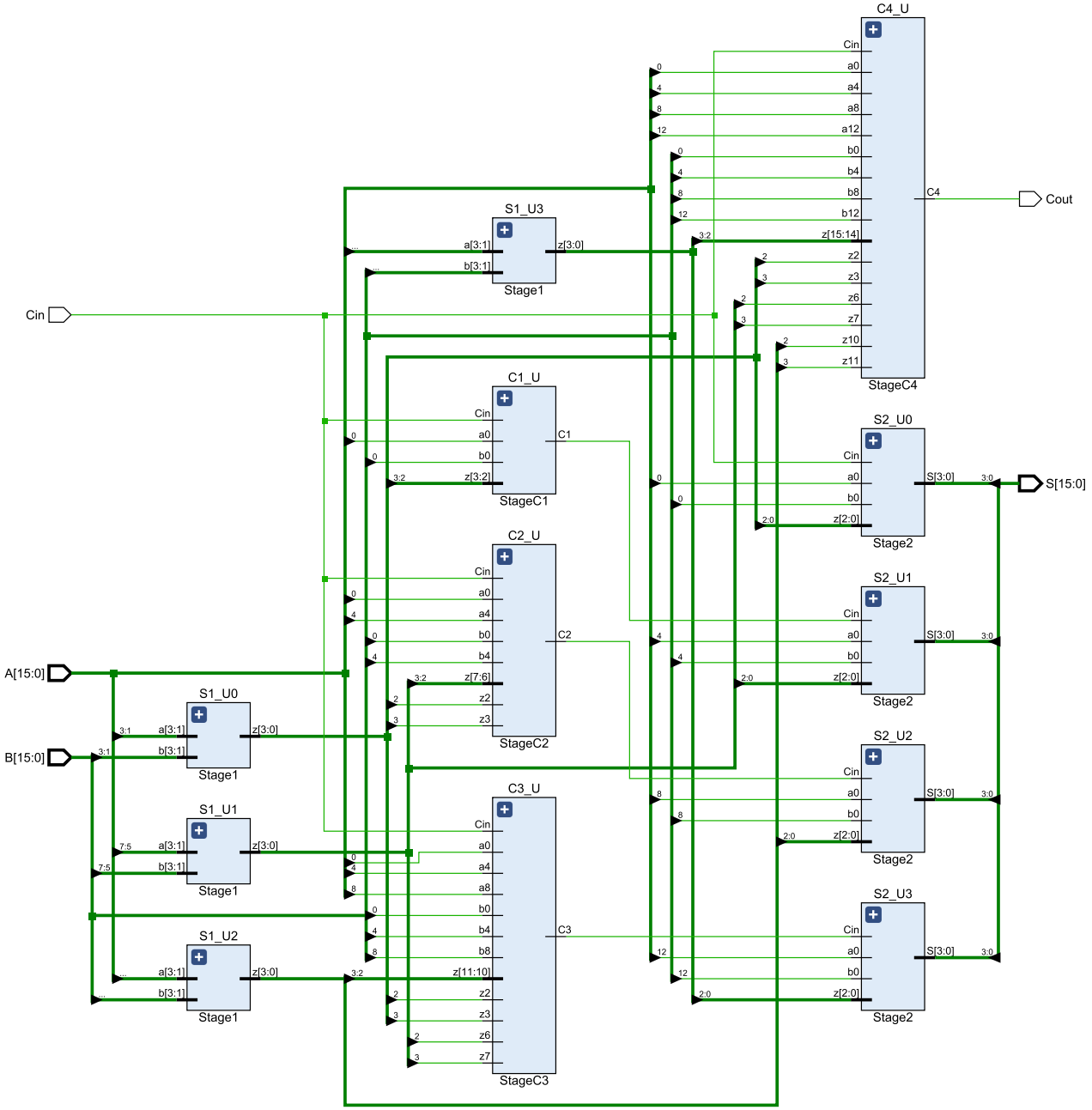


FIGURE 8. Elaborated design for the proposed $BCDA_g$.

operands, as in Table 9, the proposed $BCDA_g$ adder and the proposed $BCDA_h$ adder show a PDP of 20.17fj and 20.29fj, respectively, comparing to 38.56fj, 27.66fj, 26.31fj, 47.38fj, 48.53fj, and 23.14fj PDP for $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively.

In this scenario, the proposed $BCDA_g$ consumes 5.8% and 12.5% less power than $BCDA_a$ and $BCDA_d$ respectively. While the proposed $BCDA_h$ adder consumes 5.6% and 12.3% less power than $BCDA_a$ and $BCDA_d$, respectively. In contrast, the proposed $BCDA_g$ adder consumes 39.7%, 47.3%, 7.9%, and 3.2% extra power than $BCDA_b$, $BCDA_c$, $BCDA_e$, and $BCDA_f$, respectively. While the proposed $BCDA_h$ adder consumes 40.0%, 47.6%, 8.1%, and 3.4% more power than $BCDA_b$, $BCDA_c$, $BCDA_e$, and $BCDA_f$, respectively.

It can be found that the proposed $BCDA_g$ adder achieves 44.4%, 47.7%, 47.9%, 51.3%, 61.4%, and 15.4% reduction in delay, with an additional 38.5%, 106.0%, 90.6%, 13.1%, 27.3%, and 7.8% transistors count comparing to $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively. While the proposed $BCDA_h$ adder achieves 44.2%, 47.5%,

TABLE 5. Delay of all possible critical paths for 4-digit decimal adders.

Path	Delay(ps)							
	$BCDA_a$	$BCDA_b$ [60]	$BCDA_c$ [61]	$BCDA_d$ [62]	$BCDA_e$ [53]	$BCDA_f$ [50]	$BCDA_g$	$BCDA_h$
C_{in} to S_{15}	647.50	727.50	734.50	789.13	1004.25	366.10	295.81	302.48
a_0 to S_{15}	661.00	747.30	756.00	773.24	996.97	368.60	297.42	316.30
a_1 to S_{15}	637.03	728.95	702.98	757.20	972.18	467.38	395.19	396.54
a_2 to S_{15}	702.41	756.67	759.00	812.00	1025.18	462.47	393.21	387.16
a_3 to S_{15}	644.00	724.10	723.26	777.61	980.99	456.36	386.1	385.19
Max delay to S_{15}	702.41	756.67	759.00	812.00	1025.18	467.38	395.19	396.54
C_{in} to C_{out}	656.94	637.80	650.94	710.96	906.56	327.12	219.1	215.74
a_0 to C_{out}	668.64	657.16	671.48	693.02	900.45	329.55	220.15	230.04
a_1 to C_{out}	647.67	639.55	619.59	679.07	874.38	428.15	318.42	308.17
a_2 to C_{out}	711.33	666.27	674.48	732.92	928.14	423.15	317.11	300.93
a_3 to C_{out}	653.92	632.91	638.67	699.16	884.24	417.33	310.4	298.86
Max delay to C_{out}	711.33	666.27	674.48	732.92	928.14	428.15	318.42	308.17

TABLE 6. Comparisons against 1-digit decimal adders.

Decimal Adder	Power (μW)	Delay (ps)	Transis. Count	Pow. Del. Product (fJ)
$BCDA_a$	13.417	233.07	366	3.13
$BCDA_b$ [60]	9.043	236.99	246	2.14
$BCDA_c$ [61]	8.365	249.42	266	2.09
$BCDA_d$ [62]	14.441	242.33	448	3.50
$BCDA_e$ [53]	11.515	284.73	398	3.28
$BCDA_f$ [50]	13.012	232.19	470	3.02
$BCDA_g$	12.954	232.19	474	3
$BCDA_h$	13.133	232.18	478	3.04

TABLE 7. Comparisons against 2-digit decimal adders.

Decimal Adder	Power (μW)	Delay (ps)	Transis. Count	Pow. Del. Product (fJ)
$BCDA_a$	27.16	449.22	732	12.20
$BCDA_b$ [60]	18.216	416.14	492	7.58
$BCDA_c$ [61]	17.125	419.81	532	7.19
$BCDA_d$ [62]	29.072	444.1	896	12.91
$BCDA_e$ [53]	23.37	545.09	796	12.74
$BCDA_f$ [50]	25.175	276.61	940	6.96
$BCDA_g$	25.592	279.28	972	7.14
$BCDA_h$	25.845	266.29	968	6.88

TABLE 8. Comparisons against 3-digit decimal adders.

Decimal Adder	Power (μW)	Delay (ps)	Transis. Count	Pow. Del. Product (fJ)
$BCDA_a$	40.927	620.2	1098	25.38
$BCDA_b$ [60]	27.386	584.52	738	16.01
$BCDA_c$ [61]	25.893	588.63	798	15.24
$BCDA_d$ [62]	43.713	628.94	1344	27.49
$BCDA_e$ [53]	35.351	785.62	1194	27.77
$BCDA_f$ [50]	37.337	371.75	1410	13.88
$BCDA_g$	38.25	337.34	1490	12.9
$BCDA_h$	38.844	330.90	1470	12.85

TABLE 9. Comparisons against 4-digit decimal adders.

Decimal Adder	Power (μW)	Delay (ps)	Transis. Count	Pow. Del. Product (fJ)
$BCDA_a$	54.204	711.33	1464	38.56
$BCDA_b$ [60]	36.55	756.67	984	27.66
$BCDA_c$ [61]	34.661	759.00	1064	26.31
$BCDA_d$ [62]	58.355	812.00	1792	47.38
$BCDA_e$ [53]	47.334	1025.18	1592	48.53
$BCDA_f$ [50]	49.5	467.38	1880	23.14
$BCDA_g$	51.062	395.19	2028	20.17
$BCDA_h$	51.173	396.54	1984	20.29

47.7%, 51.1%, 61.3%, and 15.1% reduction in delay, with an additional 35.5%, 101.6%, 86.4%, 10.7%, 24.6%, and 5.5% transistors count comparing to $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively.

The relevance of the findings presented in Tables 6, 7, 8, and 9 is demonstrated by Figures 9, 10, and 11.

It should be pointed out that the proposed designs are validated using pre-layout simulations to assess their functional correctness and performance metrics. While the results are promising, post-layout simulations are necessary to account for the effects of parasitics and interconnect delays, which could influence the overall performance of the circuits. Design Rule Check (DRC) ensures that the designs comply with the fabrication process constraints, while Layout Versus Schematic (LVS) verification ensures consistency between the schematic and layout. These steps can be included in future work to further validate the practicality of the proposed adders in real-world implementations.

One more point to be clarified is that this study utilizes a single V_{th} configuration for all transistors in the 45 nm

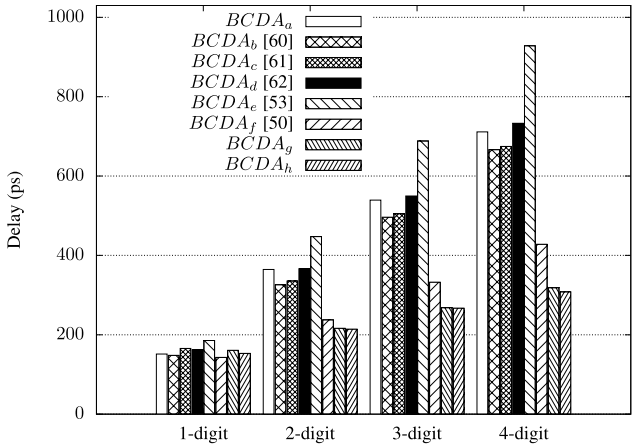


FIGURE 9. Critical path Delay from inputs to C_{out} for different decimal adders.

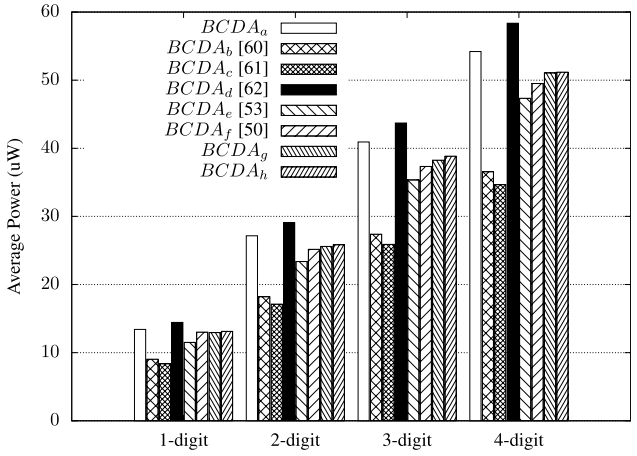


FIGURE 10. Power Consumption in different decimal adders.

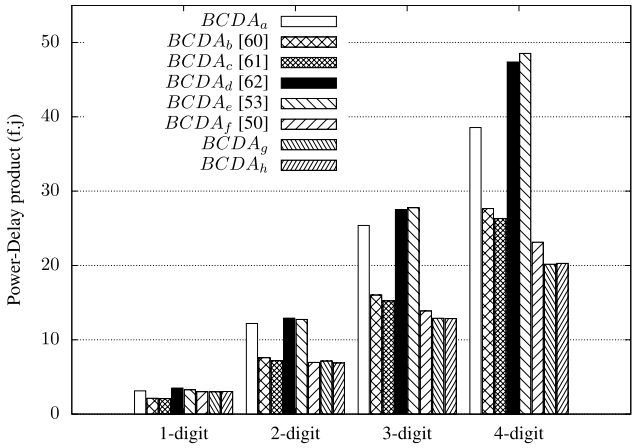


FIGURE 11. *PDP* for different decimal adders.

CMOS technology. While mixed V_{th} transistors are not explored, their use could provide additional opportunities for optimizing the power-delay product (PDP) by balancing leakage power and performance.

VI. CONCLUSION

Decimal arithmetic is receiving increased research attention due to its role in human-centric applications. Hardware implementations are preferred over software due to better performance, especially for real-time use. Decimal addition is fundamental, as other operations can be derived from it. CMOS technology helps lower power consumption and propagation delay in digital systems. Decimal adders can use either a ripple carry adder (RCA) or a carry look-ahead adder (CLA), with the latter being faster but more costly. In this work, two CMOS-based carry look-ahead BCD decimal adders (CLDAs) designed for use in any decimal arithmetic units. We have explored and investigated their functionality across operand sizes ranging from 1 to 4 digits. Detailed Boolean equations for each adder are derived, and their logic is rigorously tested and validated. The transistor-level schematics and CMOS realizations of the adders are provided. The circuit of each of the proposed CLDAs is simulated with 45 *nm* technology library using the LT-SPICE spice simulation tool. For 4-digit operands, the proposed $BCDA_g$ and $BCDA_h$ adders show a PDP of 20.17fj and 20.29fj, respectively, compared to 38.56fj, 27.66fj, 26.31fj, 47.38fj, 48.53fj, and 23.14fj for $BCDA_a$, $BCDA_b$, $BCDA_c$, $BCDA_d$, $BCDA_e$, and $BCDA_f$, respectively. $BCDA_g$ consumes 5.8% and 12.5% less power than $BCDA_a$ and $BCDA_d$, while $BCDA_h$ uses 5.6% and 12.3% less. However, $BCDA_g$ uses 39.7%, 47.3%, 7.9%, and 3.2% more power than $BCDA_b$, $BCDA_c$, $BCDA_e$, and $BCDA_f$, respectively, and $BCDA_h$ uses 40.0%, 47.6%, 8.1%, and 3.4% more than the same, respectively. $BCDA_g$ achieves 44.4% to 61.4% delay reduction with increased transistor counts of 38.5% to 106.0% compared to the other adders. $BCDA_h$ achieves 44.2% to 61.3% delay reduction with 35.5% to 101.6% more transistors.

REFERENCES

[1] W. Liu, L. Lu, M. áire, and E. E. Swartzlander, "Cost-efficient decimal adder design in quantum-dot cellular automata," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2012, pp. 1347–1350.
[2] Z. Chu, Z. Li, Y. Xia, L. Wang, and W. Liu, "BCD adder designs based on three-input XOR and majority gates," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 6, pp. 1942–1946, Jun. 2021.
[3] M. F. Cowlishaw, "Decimal floating-point: Algorism for computers," in *Proc. 16th IEEE Symp. Comput. Arithmetic*, Jun. 2003, pp. 104–111.
[4] R. D. Kenney, M. J. Schulte, and M. A. Erle, "A high-frequency decimal multiplier," in *Proc. IEEE Int. Conf. Comput. Design, VLSI Comput. Processors (ICCD)*, Oct. 2004, pp. 26–29.
[5] S. Gorgin, H. Z. Nejad, and Jeong-A. Lee, "A practical energy/power reduction approach for parallel decimal multiplier," *IEEE Access*, vol. 10, pp. 11372–11381, 2022.
[6] M. F. Mudawar, "Exact versus inexact decimal floating-point numbers and arithmetic," *IEEE Access*, vol. 11, pp. 17891–17905, 2023.
[7] G. Jaberipur and F. Ghazanfari, "Impact of radix-10 redundant digit set [−6,9] on basic decimal arithmetic operations," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 1, pp. 51–59, Jan. 2022.
[8] A. A. Wahba and H. A. H. Fahmy, "Area efficient and fast combined binary/decimal floating point fused multiply add unit," *IEEE Trans. Comput.*, vol. 66, no. 2, pp. 226–239, Feb. 2017.
[9] H. A. H. Fahmy, R. Raafat, A. M. Abdel-Majeed, R. Samy, T. ElDeeb, and Y. Farouk, "Energy and delay improvement via decimal floating point units," in *Proc. 19th IEEE Symp. Comput. Arithmetic*, Jun. 2009, pp. 221–224.

[10] *IEEE Standard for Floating-Point Arithmetic*, IEEE Standard 754-2008, 2008, pp. 1–70.

[11] *IEEE Approved Draft Standard for Floating-point Arithmetic*, IEEE Standard P754/D2.50, Apr. 2019, pp. 1–83.

[12] *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Standard 754-1985, 1985, pp. 1–20.

[13] M. Al-Khaleel, A. E. Ruchli, and M. J. Gander, "Optimized waveform relaxation methods for longitudinal partitioning of transmission lines," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 56, no. 8, pp. 1732–1743, Aug. 2009.

[14] F. Y. Busaba, C. A. Krygowski, W. H. Li, E. M. Schwarz, and S. R. Carlough, "The IBM z900 decimal arithmetic unit," in *Proc. Conf. Rec. 35th Asilomar Conf. Signals, Syst. Comput.*, vol. 2, 2001, pp. 1335–1339.

[15] L. Eisen, J. W. Ward, H.-W. Tast, N. Mading, J. Leenstra, S. M. Mueller, C. Jacobi, J. Preiss, E. M. Schwarz, and S. R. Carlough, "IBM POWER6 accelerators: VMX and DFU," *IBM J. Res. Develop.*, vol. 51, no. 6, pp. 1–21, Nov. 2007.

[16] C. F. Webb, "IBM z10: The next-generation mainframe microprocessor," *IEEE Micro*, vol. 28, no. 2, pp. 19–29, Mar. 2008.

[17] S. Carlough, A. Collura, S. Mueller, and M. Kroener, "The IBM ZEnterprise-196 decimal floating-point accelerator," in *Proc. IEEE 20th Symp. Comput. Arithmetic*, Jul. 2011, pp. 139–146.

[18] T. Yoshida, T. Maruyama, Y. Akizuki, R. Kan, N. Kiyota, K. Ikenishi, S. Itou, T. Watahiki, and H. Okano, "Sparc64 X: Fujitsu's new-generation 16-core processor for unix servers," *IEEE Micro*, vol. 33, no. 6, pp. 16–24, Nov. 2013.

[19] M. S. Schmookler and A. Weinberger, "High speed decimal addition," *IEEE Trans. Comput.*, vols. C–20, no. 8, pp. 862–866, Aug. 1971.

[20] A. Vázquez and E. Antelo, "Conditional speculative decimal addition," in *Proc. 7th Conf. Real Numbers Comput. (RNC)*, 2016, pp. 47–57.

[21] K. N. Vijeyakumar, V. Sumathy, A. D. Babu, S. Elango, and S. Saravanakumar, "FPGA implementation of low power hardware efficient flagged binary coded decimal adder," *Int. J. Comput. Appl.*, vol. 46, no. 14, pp. 41–45, May 2012.

[22] S. Mishra and G. Verma, "Low power and area efficient implementation of BCD adder on FPGA," in *Proc. Int. Conf. SIGNAL Process. Commun. (ICSC)*, Dec. 2013, pp. 461–465.

[23] M. Ul Haque, Z. T. Sworna, H. M. Hasan Babu, and A. K. Biswas, "A fast FPGA-based BCD adder," *Circuits, Syst., Signal Process.*, vol. 37, no. 10, pp. 4384–4408, Oct. 2018.

[24] N. Saravanakumar, K. S. Sudhan, K. N. Vijeyakumar, and S. Saranya, "Design and implementation of reduced power energy efficient binary coded decimal adder," *Int. J. Reconfigurable Embedded Syst. (IJRES)*, vol. 8, no. 3, p. 185, Nov. 2019.

[25] A. A.-R. Al-Nounou, O. Al-Khaleel, F. Obeidat, and M. Al-Khaleel, "FPGA implementation of fast binary multiplication based on customized basic cells," *J. Universal Comput. Sci.*, vol. 28, no. 10, pp. 1030–1057, Oct. 2022.

[26] B. A. Issa and I. S. Al-Furati, "High precision binary coded decimal (BCD) unit for 128-bit addition," in *Proc. Int. Conf. Electr., Commun., Comput. Eng. (ICECCE)*, Jun. 2020, pp. 1–5.

[27] O. Al-Khaleel, Z. Al-Qudah, M. Al-Khaleel, R. Bani-Hani, C. Papachristou, and F. Wolff, "Efficient hardware implementations of binary-to-BCD conversion schemes for decimal multiplication," *J. Circuits, Syst. Comput.*, vol. 24, no. 2, Feb. 2015, Art. no. 1550019.

[28] J. S. V. Sai Prasanthi and Y. Y. Devi, "High speed 128-bit BCD adder architecture using CLA," *Int. J. Adv. Res. Electr., Electron. Instrum. Eng.*, vol. 5, no. 10, pp. 7817–7823, Oct. 2016.

[29] O. Al-Khaleel, Z. Al-Qudah, M. Al-Khaleel, and C. Papachristou, "High performance FPGA-based decimal-to-binary conversion schemes for decimal arithmetic," *Microprocessors Microsystems*, vol. 37, no. 3, pp. 287–298, May 2013.

[30] T. Tavakoli, N. Parvin, S. Nikkhahsani, and S. R. Talebiyan, "Design of addition/subtraction for BIN/BCD numbers," *Int. J. Eng. Technol.*, vol. 9, no. 1, pp. 67–70, Feb. 2017.

[31] M. S. Deepika and M. Babu, "Design of high speed BCD adder using LUT," *Int. J. VLSI Design Commun. Syst. (IJVDCS)*, vol. 2, pp. 285–289, 2014.

[32] O. Al-Khaleel, M. Al-Khaleel, Z. Al-QudahJ, C. A. Papachristou, K. Mhaidat, and F. G. Wolff, "Fast binary/decimal adder/subtractor with a novel correction-free BCD addition," in *Proc. 18th IEEE Int. Conf. Electron., Circuits, Syst.*, Dec. 2011, pp. 455–459.

[33] O. Al-Khaleel, Z. Al-Qudah, M. Al-Khaleel, C. A. Papachristou, and F. G. Wolff, "Fast and compact binary-to-BCD conversion circuits for decimal multiplication," in *Proc. IEEE 29th Int. Conf. Comput. Design (ICCD)*, Oct. 2011, pp. 226–231.

[34] H.-Y. Lo, "A compromise between speed and complexity in a BCD adder," *Int. J. Electron.*, vol. 46, no. 1, pp. 75–79, Jan. 1979.

[35] Y. D. Ykuntam and S. H. Prasad, "A modified high speed and less area BCD adder architecture using mirror adder," in *Proc. 2nd Int. Conf. Smart Electron. Commun. (ICOSEC)*, Oct. 2021, pp. 624–627.

[36] N. Saravanakumar, K. Kumar, K. Sakthisudhan, and S. Saranya, "Design and implementation of low power energy efficient binary coded decimal adder," *Int. J. Eng. Adv. Technol. (IJEAT)*, vol. 8, pp. 119–125, Nov. 2018.

[37] R. Jain, K. Singh, and G. Jangid, "A 64 bit pipeline based decimal adder using a new high speed BCD adder," *Int. J. Eng. Adv. Technol. (IJEAT)*, vol. 3, pp. 127–131, Nov. 2014.

[38] C. Sundaresan, C. Chaitanya, J. M. Kumar, P. Venkateswaran, and S. Bhat, "Novel 16-bit and 32-bit group high speed BCD adders," *Int. J. Comput. Appl.*, vol. 107, no. 12, pp. 36–38, Dec. 2014.

[39] G. Ragunath and R. Sakthivel, "A high-speed BCD adder using single 4-bit binary adder and detector circuit," *Int. J. Eng. Adv. Technol.*, vol. 8, no. 5, pp. 1994–1999, 2019.

[40] A. N. Jayanthi and C. S. Ravichandran, "Design of a high speed BCD adder," *Eur. J. Sci. Res.*, vol. 60, pp. 422–427, Nov. 2011.

[41] Z. T. Sworna, M. UlHaque, N. Tara, H. M. Hasan Babu, and A. K. Biswas, "Low-power and area efficient binary coded decimal adder design using a look up table-based field programmable gate array," *IET Circuits, Devices Syst.*, vol. 10, no. 3, pp. 163–172, May 2016.

[42] A. Agrawal and D. P. Chaturvedi, "Area efficient high speed BCD adders using modified carry select adder," *Int. J. Emerg. Technol. Innov. Res.*, vol. 9, pp. g817–g821, May 2022.

[43] A. A. Bayrakci and A. Akkas, "Reduced delay BCD adder," in *Proc. IEEE Int. Conf. Appl.-Specific Syst., Archit. Processors (ASAP)*, Jul. 2007, pp. 266–271.

[44] P. Chaudhari, A. Khadke, M. More, and P. D. S. Patil, "Ultra low power, low voltage 16 bit BCD adder using dtmos technique," *Int. Res. J. Eng. Technol. (IRJET)*, vol. 3, pp. 1115–1118, Oct. 2016.

[45] R. Santhiya and T. ThamaraiManalan, "Power gating based low power 32 bit BCD adder using DVT," *Int. J. Sci. Res. Develop.*, vol. 3, no. 2, pp. 802–805, 2015.

[46] S. Goswami, A. Mandal, A. Mishra, J. Goswami, and A. Saha, "Novel CMOS 1-digit BCD-adder correction circuit," in *Proc. Int. Conf. Commun., Devices Comput.* Cham, Switzerland: Springer, Mar. 2023, pp. 189–195.

[47] D. Siddhamshittiwar, "An efficient power optimized 32 bit BCD adder using multi-channel technique," *Int. J. New Practices Manage. Eng.*, vol. 6, no. 2, pp. 7–12, Jun. 2017.

[48] D. Saha, S. Basak, S. Mukherjee, and C. K. Sarkar, "A low-voltage, low-power 4-bit BCD adder, designed using the clock gated power gating, and the DVT scheme," in *Proc. IEEE Int. Conf. Signal Process., Comput. Control (ISPCC)*, Sep. 2013, pp. 1–6.

[49] B. Shiby and P. Reen, "Study on a compact and high speed 4-bit BCD adder," *J. Electron. Design Eng.*, vol. 2, pp. 1–11, May 2016.

[50] A. A. Share, F. N. Zghoul, O. Al-Khaleel, M. Al-Khaleel, and C. Papachristou, "Design of high speed BCD adder using CMOS technology," *IEEE Access*, vol. 11, pp. 141628–141639, 2023.

[51] S. Ghafari, M. Mousazadeh, A. Khoei, and A. Dadashi, "A new high-speed and low area efficient pipelined 128-bit adder based on modified carry look-ahead merging with han-carlson tree method," in *Proc. MIXDES - 26th Int. Conf. 'Mixed Design Integr. Circuits Syst.'*, Jun. 2019, pp. 157–162.

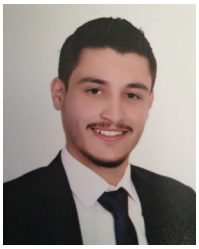
[52] I. Koren, *Computer Arithmetic Algorithms*. Boca Raton, FL, USA: CRC Press, 2018.

[53] M. Hasan, M. S. Hossain, A. H. Siddique, M. Hossain, H. U. Zaman, and S. Islam, "A high-speed 4-bit carry look-ahead architecture as a building block for wide word-length carry-select adder," *Microelectron. J.*, vol. 109, Mar. 2021, Art. no. 104992.

[54] R. Zlatanovici, S. Kao, and B. Nikolic, "Energy–delay optimization of 64-bit carry-lookahead adders with a 240 ps 90 nm CMOS design example," *IEEE J. Solid-State Circuits*, vol. 44, no. 2, pp. 569–583, Feb. 2009.

[55] M. Valinataj, "Fault-tolerant carry look-ahead adder architectures robust to multiple simultaneous errors," *Microelectron. Rel.*, vol. 55, no. 12, pp. 2845–2857, Dec. 2015.

[56] A. Ibrahim and F. Gebali, "Optimized structures of hybrid ripple carry and hierarchical carry lookahead adders," *Microelectron. J.*, vol. 46, no. 9, pp. 783–794, Sep. 2015.
[57] M. S. Hossain and F. Arifin, "A proposed design of conventional 4-bit carry look-ahead adder improving performance," in *Proc. Adv. Comput. Commun. Technol. High Perform. Appl. (ACCTHPA)*, Jul. 2020, pp. 89–93.
[58] M. Binggeli, S. Denton, N. S. Muppaneni, and S. Chiu, "Optimizing carry-lookahead logic through a comparison of PMOS and NMOS block inversions," in *Proc. IEEE Int. Conf. Electro/Inf. Technol. (EIT)*, May 2015, pp. 641–646.
[59] M. Hasan, Md. J. Hossein, M. Hossain, H. U. Zaman, and S. Islam, "Design of a scalable low-power 1-bit hybrid full adder for fast computation," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 8, pp. 1464–1468, Aug. 2020.
[60] M. Hasan, M. Sadia Islam, and M. Rafid Ahmed, "Performance improvement of 4-Bit static CMOS carry look-ahead adder using modified circuits for carry propagate and generate terms," *Sci. J. Circuits, Syst. Signal Process.*, vol. 8, no. 2, p. 76, 2019.
[61] G. A. Ruiz and M. Granda, "An area-efficient static CMOS carry-select adder based on a compact carry look-ahead unit," *Microelectron. J.*, vol. 35, no. 12, pp. 939–944, Dec. 2004.
[62] J. Miao and S. Li, "A novel implementation of 4-bit carry look-ahead adder," in *Proc. Int. Conf. Electron Devices Solid-State Circuits (EDSSC)*, Oct. 2017, pp. 1–2.
[63] C. H. Roth and L. L. Kinney, *Fundamentals of Logic Design*, 7th ed., Boston, MA, USA: Cengage, 2014.



ABDELSALAM AL SHARE received the B.Sc. degree in electrical engineering from Yarmouk University, in 2018, and the M.Sc. degree in electrical engineering from Jordan University of Science and Technology, Jordan, in 2023. He did practical training for three months with Irbid Electricity Company, from June 2017 to September 2017. He was an Electrical Engineer with Jordan Petroleum Refinery, from September 2018 to February 2019. Currently, he is with the Ministry of Education, Jordan. His current research interests include electrical circuit design, electrical distribution systems, and electronic circuit design.

OSAMA AL-KHALEEL received the B.Sc. degree in electrical engineering from Jordan University of Science and Technology, in 1999, and the M.Sc. and Ph.D. degrees in computer engineering from Case Western Reserve University, Cleveland, OH, USA, in 2003 and 2006, respectively. He is currently an Associate Professor of computer engineering with the Department of Computer Engineering, Jordan University of Science and Technology, Irbid, Jordan. His current main research interests include embedded systems design, reconfigurable computing, computer arithmetic, and logic design.

FADI NESSIR ZGHOUL received the M.Sc. and Ph.D. degrees in electrical engineering from the University of Idaho, ID, USA, in 2003 and 2007, respectively. In 2007, he joined The Hashemite University, Jordan, as an Assistant Professor. After that, he joined Yarmouk University, Irbid, Jordan, in 2008, also as an Assistant Professor. Since 2009, he has been appointed as the Dean Assistant of the Hijjawi Faculty for Engineering Technology, Yarmouk University. Since 2010, he has been an Associate Professor with Jordan University for Science and Technology, Irbid, Jordan. Since 2014, he has been appointed as the Dean Assistant of the Faculty of Engineering, Jordan University for Science and Technology. In 2019, he spent one academic year with The University of Texas at Austin as a Visiting Scholar, sponsored by Fulbright. His current research interests include mixed signal circuit design, power management systems, and improving simulation techniques for nonlinear circuits.

MOHAMMAD AL-KHALEEL received the M.Sc. and Ph.D. degrees in applied mathematics (numerical analysis and scientific computing) from McGill University, Montreal, QC, Canada, in 2003 and 2007, respectively. In 2007, he became an Assistant Professor of mathematics with the Department of Mathematics, Yarmouk University, Jordan, and afterwards was promoted to an Associate Professor. He is currently an Associate Professor with the Department of Mathematics, Khalifa University, United Arab Emirates. His research is of a multidisciplinary nature but the bulk of it has revolved around developing and implementing efficient and parallel numerical methods as well as analytical methods for solving differential equations (ordinary, partial, and fractional) that appear in many applications in different fields, including seismic waves, nanofluid and microfluid dynamics, and circuit simulations. He has also research interests in other areas of mathematics, including topics in metric fixed point theory, fractional calculus, numerical parameter optimization, inverse and control problems, and the mathematical background of computer arithmetic circuits.

CHRIS PAPACHRISTOU (Life Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer science from the Johns Hopkins University, Baltimore, MD, USA. He is currently a Professor of electrical and computer engineering with Case Western Reserve University, Cleveland, OH, USA. His research interests include embedded systems, fault-tolerant, and secure system design, VLSI CAD, and reconfigurable computing.